/* ========== MASA ARXASINDA GAME.JS — 1-Cİ HİSSƏ
Bu hissə:
1. Firebase bağlantısı
2. Anonim istifadəçi girişi
3. Otaq yaratmaq və qoşulmaq
4. Oyunçu və personaj sistemi
5. Hazır olma sistemi
6. Raund və kart seçimi
7. Otaq məlumatlarının sinxronlaşdırılması
2-ci hissə bu faylın SONUNA əlavə ediləcək.
========== */
/* ==========
1. FIREBASE IMPORTLARI
========== */
import { initializeApp } from "https://www.gstatic.com/firebasejs/12.6.0/firebase-app.js";
import { getAuth, signInAnonymously, onAuthStateChanged } from "https://www.gstatic.com/firebasejs/12.6.0/firebase-auth.js";
import { getDatabase, ref, set, get, update, onValue, runTransaction, remove, onDisconnect } from "https://www.gstatic.com/firebasejs/12.6.0/firebase-database.js";
/* ========== 2. FIREBASE KONFİQURASİYASI ========== */
// For Firebase JS SDK v7.20.0 and later, measurementId is optional const firebaseConfig = { apiKey: "AIzaSyBFFVigbO3r5vZbl7vfgzVboChHjXn2h-A", authDomain: "behind-table.firebaseapp.com", databaseURL: "https://behind-table-default-rtdb.firebaseio.com", projectId: "behind-table", storageBucket: "behind-table.firebasestorage.app", messagingSenderId: "637325725035", appId: "1:637325725035:web:696bb4b800ed5ba713d02e", measurementId: "G-EJN8J5HFGD" }; const app = initializeApp(firebaseConfig); const auth = getAuth(app); const db = getDatabase(app);
/* ========== 3. ƏSAS DƏYİŞƏNLƏR ========== */
let uid = null; let currentRoom = null; let roomData = null;
let roomUnsub = null; let authReady = false; let busy = false;
let selectedCards = []; let selectedTarget = null;
let lastEventId = null; let lastEffectId = null;
let isPracticeMode = false; let practiceBots = [];
let toastTimeout = null;
const $ = (id) => document.getElementById(id);
/* ========== 4. OYUNÇU PERSONAJLARI
Hər personajın öz siması var. Siksoran adı Əməsgəri ilə əvəz edilib.
========== */
const CHARACTERS = [{ id: "moto", name: "Moto moto", face: "😎" }, { id: "polat", name: "Polat Amımdar", face: "🕶️" }, { id: "lez", name: "Lez Ziya", face: "🤠" }, { id: "memesbas", name: "Məməşbaş", face: "🤓" }, { id: "emdul", name: "Əmdüley", face: "🥸" }, { id: "maygulu", name: "Maygülü", face: "😈" }, { id: "emesgeri", name: "Əməsgəri", face: "👽" }, { id: "totes", name:

"Tötəş", face: "🤡" }, { id: "amir", name: "Əmir", face: "🧐" }, { id: "chuban", name: "Çuban Ələddin", face: "🤖" }];
/* ========== 5. KART MƏLUMATLARI
Kartların mexanikaları 2-ci hissədə işləyəcək. Burada onların seçilməsi və saxlanması qurulur.
========== */
const CARD_DATA = { detective: { id: "detective", name: "Detektiv", icon: "🔎", description: "Növbəti kamerada güllə olub-olmadığını öyrən.", color: "#48c6ff" },
swap: { id: "swap", name: "Yerdəyişmə", icon: "🔄", description: "Sənə atəş açıldıqda zərbəni hücumçuya yönəlt.", color: "#54e39a" },
counter: { id: "counter", name: "Əks təsir", icon: "🛡️", description: "Dolu güllə səni vurarsa, zərəri blokla və can qazan.", color: "#8b5cf6" },
witch: { id: "witch", name: "İfritəlik", icon: "🧙", description: "Seçilmiş hədəfə növbəti atəşin vurma şansı 80% olsun.", color: "#ff5caa" },
selfLove: { id: "selfLove", name: "Canını sevmək", icon: "❤️", description: "Özünə atəş açanda vurulma ehtimalını 1/4-ə endir.", color: "#ff5364" },
evil: { id: "evil", name: "İbnəlik", icon: "☠️", description: "Dolu güllə hədəfi dərhal öldürür; boş çıxarsa, özün 1 can itirirsən.", color: "#ffd447" } };
/* ========== 6. RAUND QAYDALARI ========== */
const ROUND_CONFIG = { 1: { chambers: 6, bullets: 2, cards: 1, selectionTime: 30, automaticCards: false },
2: { chambers: 8, bullets: 3, cards: 2, selectionTime: 30, automaticCards: false },
3: { chambers: 9, bullets: 4, cards: 2, selectionTime: 30, automaticCards: false },
4: { chambers: 9, bullets: 5, cards: 3, selectionTime: 30, automaticCards: false },
5: { chambers: 12, bullets: 7, cards: 3, selectionTime: 0, automaticCards: true } };
/* ========== 7. ÜMUMİ KÖMƏKÇİ FUNKSİYALAR
========== */
function randomId(length = 12) { const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789";
let result = "";
for (let i = 0; i < length; i++) { result += chars[Math.floor(Math.random() * chars.length)]; }
return result; }
function makeRoomCode() { const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
let code = "";
for (let i = 0; i < 4; i++) { code += chars[Math.floor(Math.random() * chars.length)]; }
return code; }
function randomItem(array) { return array[Math.floor(Math.random() * array.length)]; }
function escapeHTML(value) { return String(value ?? "").replace(/[&<>"']/g, (char) => { const replacements = { "&": "&", "<": "<", ">": ">", "": """, "": "'" };
return replacements[char]; }); }
function getCharacter(characterId) { return CHARACTERS.find(character => character.id === characterId) || CHARACTERS[0]; }
function getPlayerName(player) { return player?.name || "Naməlum oyunçu"; }
function getRoundConfig(round) { return ROUND_CONFIG[round] || ROUND_CONFIG[5]; }
function setText(id, value) { const element = $(id);
if (element) { element.textContent = value; } }

```
function showScreen(screenId) { document.querySelectorAll(".screen").forEach(screen => {
screen.classList.remove("active"); });
const target = $(screenId);
if (target) { target.classList.add("active"); } }
function setBusy(value) { busy = value;
document.querySelectorAll("button").forEach(button => { if (button.dataset.ignoreBusy ===
"true") return;
button.disabled = value; }); }
function showToast(message) { const toast = $("toast");
if (!toast) { console.log(message); return; }
toast.textContent = message; toast.classList.add("show");
clearTimeout(toastTimeout);
toastTimeout = setTimeout(() => { toast.classList.remove("show"); }, 2600); }
function showError(message) { const error = $("homeError");
if (error) { error.textContent = message; } else { showToast(message); } }
function clearError() { const error = $("homeError");
if (error) { error.textContent = ""; } }
function getNickname() { const input = $("nickname");
const name = input?.value?.trim() || "";
if (name.length < 2) { throw new Error("Ləqəb ən azı 2 simvoldan ibarət olmalıdır."); }
if (name.length > 18) { throw new Error("Ləqəb maksimum 18 simvol ola bilər."); }
return name; }
function requireAuth() { if (!authReady || !uid) { throw new Error("Serverə qoşulma gözlənilir. Bir
az sonra yenidən yoxla."); } }
function getRoomRef() { if (!currentRoom) { throw new Error("Hazırda otaqda deyilsən."); }
return ref(db, rooms/${currentRoom}); }
function getMyPlayer() { if (!roomData || !uid) return null;
return roomData.players?.[uid] || null; }
function isHost() { return roomData?.host === uid; }
function getPlayers() { if (!roomData?.players) return [];
return Object.entries(roomData.players).map(([id, player]) => ({ ...player, uid: id })); }
function getAlivePlayers() { return getPlayers().filter(player => player.alive !== false); }
function getSelectedCardLimit() { const round = Number(roomData?.round || 1);
return getRoundConfig(round).cards; }
/* ========================================================== 8. HADİSƏ
SİSTEMİ ========================================================== */
function createEvent(message, type = "normal") { return { id: randomId(16), message, type,
time: Date.now() }; }
function addRoomEvent(message, type = "normal") { if (!currentRoom) return Promise.resolve();
const event = createEvent(message, type); const events = Array.isArray(roomData?.events) ?
[...roomData.events] : [];
events.push(event);
return update(getRoomRef(), { events: events.slice(-60) }); }
function renderEventLog(events = []) { const log = $("eventLog");
if (!log) return;
log.innerHTML = "";
[...events].reverse().forEach(event => { const entry = document.createElement("div");
entry.className = event-entry event-${event.type || "normal"}; entry.textContent =
```

```
event.message || "";
log.appendChild(entry); }); }
function renderLatestEvent(events = []) { const latest = $("latestEvent");
if (!latest || !events.length) return;
const event = events[events.length - 1];
if (!event || event.id === lastEventId) return;
lastEventId = event.id; latest.textContent = event.message || "Oyun davam edir...";
latest.classList.remove("event-update");
// Animasiya yenidən başlasın. void latest.offsetWidth;
latest.classList.add("event-update"); }
/* ========================================================== 9.
AUTHENTICATION — ANONİM GİRİŞ
========================================================== */
onAuthStateChanged(auth, async (user) => { if (user) { uid = user.uid; authReady = true;
const connection = $("connection");
if (connection) { connection.textContent = "● SERVERƏ BAĞLI"; connection.style.color =
"#54e39a"; }
console.log("Firebase istifadəçisi hazırdır:", uid);
} else { authReady = false;
const connection = $("connection");
if (connection) { connection.textContent = "● BAĞLANTI GÖZLƏNİLİR"; connection.style.color =
"#ffd447"; }
try { await signInAnonymously(auth); } catch (error) { console.error("Anonim giriş xətası:", error);
showToast("Serverə qoşulmaq mümkün olmadı."); } } });
/* ========================================================== 10. SERVER
BAĞLANTISININ İZLƏNMƏSİ
========================================================== */
const connectionRef = ref(db, ".info/connected");
onValue(connectionRef, snapshot => { const connected = snapshot.val() === true; const
connection = $("connection");
if (!connection) return;
if (connected) { connection.textContent = "● SERVERƏ BAĞLI"; connection.style.color =
"#54e39a"; } else { connection.textContent = "● BAĞLANTI KƏSİLDİ"; connection.style.color =
"#ff5364"; } });
/* ========================================================== 11. OTAQ
MƏLUMATLARINI DİNLƏMƏ
========================================================== */
function unsubscribeRoom() { if (roomUnsub) { roomUnsub(); roomUnsub = null; } }
function subscribeRoom(roomCode) { unsubscribeRoom();
currentRoom = roomCode;
const roomReference = ref(db, rooms/${roomCode});
roomUnsub = onValue( roomReference,
snapshot => { if (!snapshot.exists()) { roomData = null;
showToast("Otaq artıq mövcud deyil.");
unsubscribeRoom(); currentRoom = null;
showScreen("home"); return; }
roomData = snapshot.val();
renderRoom(); },
```

```
error => { console.error("Otaq dinləmə xətası:", error);
showToast("Otaq məlumatlarını almaq mümkün olmadı."); } ); }
/* ========== 12. OTAĞIN ÜMUMİ RENDER FUNKSİYASI ========== */
function renderRoom() { if (!roomData) return;
if (roomData.status === "lobby") { renderLobby(); return; }
if ( roomData.status === "cardSelection" || roomData.status === "card-selection" ) {
renderCardSelection();
if (typeof renderCardSelectionExtra === "function") { renderCardSelectionExtra(); }
return; }
if (roomData.status === "game") { showScreen("game");
if (typeof renderGameScreen === "function") { renderGameScreen(); }
return; }
if (roomData.status === "ended") { showScreen("ended");
if (typeof renderEndScreen === "function") { renderEndScreen(); } } }
/* ========== 13. OTAQ YARATMA ========== */
async function createRoom() { if (busy) return;
clearError();
try { requireAuth();
const name = getNickname();
setBusy(true);
let roomCode = null; let created = false;
// Eyni kodun təsadüfən təkrar düşməsinə qarşı yoxlama. for (let attempt = 0; attempt < 12;
attempt++) { const candidate = makeRoomCode();
const candidateRef = ref(db, rooms/${candidate}); const snapshot = await get(candidateRef);
if (!snapshot.exists()) { roomCode = candidate; break; } }
if (!roomCode) { throw new Error("Otaq kodu yaratmaq mümkün olmadı. Yenidən yoxla."); }
const character = CHARACTERS[0];
const newRoom = { status: "lobby",
host: uid,
createdAt: Date.now(),
round: 1,
currentTurn: uid,
players: { [uid]: { uid, name, character: character.id, face: character.face,
alive: true, hearts: 2,
ready: false, cardReady: false,
selectedCards: [], usedCards: [],
joinedAt: Date.now(), isBot: false } },
events: [ createEvent(${name} otağı yaratdı., "system") ],
gameState: null };
const roomReference = ref(db, rooms/${roomCode});
await set(roomReference, newRoom);
created = true;
if (created) { subscribeRoom(roomCode); showToast(Otaq yaradıldı: ${roomCode}); }
} catch (error) { console.error("Otaq yaratma xətası:", error);
showError(error.message || "Otaq yaratmaq mümkün olmadı.");
```

```
} finally { setBusy(false); } }
/* ========== 14. OTAĞA QOŞULMA ========== */
async function joinRoom() { if (busy) return;
clearError();
try { requireAuth();
const name = getNickname();
const code = $("roomCodeInput")?.value ?.trim() .toUpperCase();
if (!code || code.length !== 4) { throw new Error("4 simvollu otaq kodunu daxil et."); }
setBusy(true);
const roomReference = ref(db, rooms/${code}); const snapshot = await get(roomReference);
if (!snapshot.exists()) { throw new Error("Belə otaq tapılmadı."); }
const data = snapshot.val();
if (data.status !== "lobby") { throw new Error("Bu otaqda oyun artıq başlayıb."); }
if (data.players?.[uid]) { subscribeRoom(code); return; }
const playerCount = Object.keys(data.players || {}).length;
if (playerCount >= 10) { throw new Error("Otaq doludur. Maksimum 10 oyunçu ola bilər."); }
const character = CHARACTERS.find(character => { return !Object.values(data.players ||
{}).some( player => player.character === character.id ); }) || randomItem(CHARACTERS);
const newPlayer = { uid, name,
character: character.id, face: character.face,
alive: true, hearts: 2,
ready: false, cardReady: false,
selectedCards: [], usedCards: [],
joinedAt: Date.now(), isBot: false };
await update( ref(db, rooms/code/players/{uid}), newPlayer );
await update(roomReference, { events: [ ...(data.events || []), createEvent(${name} otağa
qoşuldu., "system") ].slice(-60) });
subscribeRoom(code);
showToast("Otağa uğurla qoşuldun!");
} catch (error) { console.error("Otağa qoşulma xətası:", error);
showError(error.message || "Otağa qoşulmaq mümkün olmadı.");
} finally { setBusy(false); } }
/* ========== 15. LOBBİ RENDER ========== */
function renderLobby() { showScreen("lobby");
setText("roomCodeDisplay", currentRoom || "----");
const players = getPlayers();
setText("playerCount", ${players.length}/10);
renderPlayerList(players); renderCharacterSelect();
const me = getMyPlayer();
if ($("readyStatus")) { $("readyStatus").textContent = me?.ready ? "Hazırsan!" : "Hələ hazır
deyilsən."; }
const readyButton = $("readyButton");
if (readyButton) { readyButton.textContent = me?.ready ? "Hazır deyiləm" : "Hazıram"; }
const startButton = $("startGame");
if (startButton) { startButton.style.display = isHost() ? "" : "none";
const allReady = players.length >= 2 && players.every(player => player.ready === true);
```

```
startButton.disabled = !allReady;
startButton.textContent = allReady ? "OYUNA BAŞLA" : "OYUNÇULARI GÖZLƏ"; }
const hint = $("lobbyHint");
if (hint) { hint.textContent = isHost() ? "Bütün oyunçular hazır olduqda oyunu başlada bilərsən." :
"Ev sahibinin oyunu başlatmasını gözlə."; } }
function renderPlayerList(players) { const list = $("playerList");
if (!list) return;
list.innerHTML = "";
players.forEach(player => { const character = getCharacter(player.character);
const item = document.createElement("div");
item.className = "player-item";
if (player.uid === roomData.host) { item.classList.add("host"); }
const avatar = document.createElement("span"); avatar.className = "player-avatar";
avatar.textContent = character.face;
const name = document.createElement("span"); name.className = "player-name";
name.textContent = player.name;
const status = document.createElement("span"); status.className = "host-label";
if (player.uid === roomData.host) { status.textContent = "HOST"; } else { status.textContent =
player.ready ? "HAZIR" : "GÖZLƏYİR"; status.style.color = player.ready ? "#54e39a" :
"#b7afd7"; }
item.append(avatar, name, status);
list.appendChild(item); }); }
/* ========================================================== 16. PERSONAJ
SEÇİMİ ========================================================== */
function renderCharacterSelect() { const select = $("characterSelect");
if (!select) return;
const me = getMyPlayer();
if (!me) return;
const previousValue = me.character || CHARACTERS[0].id;
select.innerHTML = "";
CHARACTERS.forEach(character => { const option = document.createElement("option");
option.value = character.id; option.textContent = ${character.face} ${character.name};
const usedByOther = getPlayers().some(player => { return player.uid !== uid &&
player.character === character.id; });
option.disabled = usedByOther;
select.appendChild(option); });
select.value = previousValue; }
async function changeCharacter(characterId) { if (!currentRoom || !uid) return;
const character = CHARACTERS.find( item => item.id === characterId );
if (!character) return;
const usedByOther = getPlayers().some(player => { return player.uid !== uid &&
player.character === characterId; });
if (usedByOther) { showToast("Bu personaj artıq seçilib."); return; }
try { await update( ref(db, rooms/currentRoom/players/{uid}), { character: character.id, face:
character.face, ready: false } );
showToast(${character.name} seçildi.);
} catch (error) { console.error("Personaj dəyişmə xətası:", error); showToast("Personajı
dəyişmək mümkün olmadı."); } }
```

```
/* ========== 17. HAZIR OLMA SİSTEMİ ========== */
async function toggleReady() { if (!currentRoom || !uid || busy) return;
const me = getMyPlayer();
if (!me) return;
try { await update( ref(db, rooms/currentRoom/players/{uid}), { ready: !me.ready } );
} catch (error) { console.error("Hazır olma xətası:", error); showToast("Status dəyişdirilə bilmədi."); } }
/* ========== 18. OYUNUN BAŞLANMASI ========== */
async function startGame() { if (busy || !currentRoom || !isHost()) return;
try { const players = getPlayers();
if (players.length < 2) { throw new Error("Oyun üçün ən azı 2 oyunçu lazımdır."); }
if (players.length > 10) { throw new Error("Maksimum 10 oyunçu ola bilər."); }
if (!players.every(player => player.ready === true)) { throw new Error("Bütün oyunçular hazır olmalıdır."); }
setBusy(true);
const initialRound = 1; const roundConfig = getRoundConfig(initialRound);
const updates = { status: "cardSelection", round: initialRound, currentTurn: null,
gameState: { round: initialRound, chambers: roundConfig.chambers, bullets:
roundConfig.bullets, shotsTaken: 0, shotLimit: null, chamberIndex: 0, chamberOrder: [],
selectedTarget: null, selectionStartedAt: Date.now(), selectionEndsAt: Date.now() +
roundConfig.selectionTime * 1000, cardSelectionsComplete: false },
events: [ ...(roomData.events || []), createEvent( "Oyun başlayır! Birinci raund: kart seçimi.",
"system" ) ].slice(-60) };
const playerUpdates = {};
players.forEach(player => { playerUpdates[players/
player.uid/ready] = false; playerUpdates[players/{player.uid}/cardReady] = false;
playerUpdates[players/player.uid/selectedCards] = []; playerUpdates[players/
{player.uid}/usedCards] = []; playerUpdates[players/
player.uid/alive] = true; playerUpdates[players/{player.uid}/hearts] = 2; });
await update(getRoomRef(), { ...updates, ...playerUpdates });
selectedCards = [];
} catch (error) { console.error("Oyuna başlama xətası:", error); showToast(error.message ||
"Oyunu başlatmaq mümkün olmadı.");
} finally { setBusy(false); } }
/* ========== 19. KART SEÇİMİ EKRANI ========== */
function renderCardSelection() { showScreen("cardSelection");
const round = Number(roomData.round || 1); const config = getRoundConfig(round); const me =
getMyPlayer();
if (!me) return;
setText("cardRound", RAUND ${round});
setText( "cardSelectionDescription", config.automaticCards ? "Bu raundda hər oyunçuya
təsadüfi 3 fərqli kart veriləcək." : Bu raund üçün ${config.cards} kart seçməlisən. );
setText( "selectedCardCount", (me.selectedCards||[]).length/{config.cards} );
setText( "cardTimer", config.automaticCards ? "AVTOMATİK SEÇİM" : "SEÇİM MÜDDƏTİ: 30
SANİYƏ" );
```

```
const grid = $("cardGrid");
if (!grid) return;
const cards = grid.querySelectorAll(".game-card"); const playerCards = me.selectedCards || [];
cards.forEach(button => { const cardId = button.dataset.card; const card =
CARD_DATA[cardId];
if (!card) return;
const selected = playerCards.includes(cardId);
button.classList.toggle("selected", selected);
button.setAttribute( "aria-pressed", selected ? "true" : "false" );
const icon = button.querySelector( ".card-image, .card-icon" );
if (icon) { icon.textContent = card.icon; }
const name = button.querySelector(".card-name");
if (name) { name.textContent = card.name; }
const description = button.querySelector( ".card-description" );
if (description) { description.textContent = card.description; }
const used = (me.usedCards || []).includes(cardId);
button.disabled = used && !selected; });
const confirmButton = $("confirmCards");
if (confirmButton) { const ready = me.cardReady === true;
confirmButton.textContent = ready ? "SEÇİM TƏSDİQLƏNDİ" : "KARTLARI TƏSDİQLƏ";
confirmButton.disabled = ready || (!config.automaticCards && playerCards.length !==
config.cards); } }
/* ========================================================== 20. KART
SEÇMƏ / SEÇİMDƏN ÇIXARMA
========================================================== */
async function toggleCard(cardId) { if (!currentRoom || !uid || busy) return;
if (!CARD_DATA[cardId]) return;
const me = getMyPlayer();
if (!me || me.cardReady) return;
const round = Number(roomData.round || 1); const config = getRoundConfig(round);
if (config.automaticCards) { showToast("Bu raundda kartlar avtomatik verilir."); return; }
const currentSelection = [ ...(me.selectedCards || []) ];
const alreadySelected = currentSelection.includes(cardId);
let newSelection;
if (alreadySelected) { newSelection = currentSelection.filter( id => id !== cardId ); } else { if
(currentSelection.length >= config.cards) { showToast( Bu raundda yalnız ${config.cards} kart
seçə bilərsən. ); return; }
if ((me.usedCards || []).includes(cardId)) { showToast("Bu kart artıq istifadə olunub."); return; }
newSelection = [...currentSelection, cardId]; }
try { await update( ref(db, rooms/currentRoom/players/{uid}), { selectedCards: newSelection }
);
} catch (error) { console.error("Kart seçmə xətası:", error); showToast("Kart seçimini dəyişmək
mümkün olmadı."); } }
/* ========================================================== 21. KARTLARIN
TƏSDİQLƏNMƏSİ ==========================================================
*/
async function confirmCards() { if (!currentRoom || !uid || busy) return;
const me = getMyPlayer();
```

```
if (!me || me.cardReady) return;
const round = Number(roomData.round || 1); const config = getRoundConfig(round);
let cards = [...(me.selectedCards || [])];
if (config.automaticCards) { cards = selectRandomCards( config.cards, me.usedCards || [] ); }
else if (cards.length !== config.cards) { showToast(Dəqiq ${config.cards} kart seçməlisən.);
return; }
try { setBusy(true);
await update( ref(db, rooms/currentRoom/players/{uid}), { selectedCards: cards, cardReady:
true } );
showToast("Kart seçimin təsdiqləndi!");
if (typeof startRoundEngine === "function") { await startRoundEngine(); }
} catch (error) { console.error("Kart təsdiqləmə xətası:", error); showToast("Kartları təsdiqləmək
mümkün olmadı.");
} finally { setBusy(false); } }
function selectRandomCards(count, excluded = []) { const available =
Object.keys(CARD_DATA).filter( id => !excluded.includes(id) );
const shuffled = [...available];
for (let i = shuffled.length - 1; i > 0; i--) { const j = Math.floor(Math.random() * (i + 1));
[shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]]; }
return shuffled.slice(0, count); }
/* ========== 22. KART
SEÇİMİNİN ÜMUMİ YOXLAMASI
========== */
function allPlayersSelectedCards() { const players = getAlivePlayers();
return players.length >= 2 && players.every(player => player.cardReady === true); }
/* ========== 23. OTAQDAN
ÇIXMA ========== */
async function leaveRoom() { if (!currentRoom || !uid) { showScreen("home"); return; }
const code = currentRoom;
try { const roomReference = ref(db, rooms/${code}); const snapshot = await
get(roomReference);
if (!snapshot.exists()) { unsubscribeRoom(); currentRoom = null; roomData = null;
showScreen("home"); return; }
const data = snapshot.val(); const players = data.players || {};
const remainingPlayers = Object.keys(players).filter( id => id !== uid );
if (remainingPlayers.length === 0) { await remove(roomReference);
} else { const updates = {};
updates[players/${uid}] = null;
if (data.host === uid) { updates.host = remainingPlayers[0];
const newHost = players[remainingPlayers[0]];
updates.events = [ ...(data.events || []), createEvent( ${newHost?.name || "Yeni oyunçu"} yeni ev
sahibi oldu., "system" ) ].slice(-60);
} else { updates.events = [ ...(data.events || []), createEvent( ${players[uid]?.name || "Oyunçu"}
otaqdan çıxdı., "system" ) ].slice(-60); }
await update(roomReference, updates); }
} catch (error) { console.error("Otaqdan çıxma xətası:", error);
} finally { unsubscribeRoom();
currentRoom = null; roomData = null;
```

```
selectedCards = []; selectedTarget = null;
isPracticeMode = false;
showScreen("home"); } }
/* ========== 24. OTAQ KODUNU KOPYALAMA ========== */
async function copyRoomCode() { if (!currentRoom) return;
try { await navigator.clipboard.writeText(currentRoom); showToast("Otaq kodu kopyalandı!");
} catch (error) { console.error("Kopyalama xətası:", error); showToast(Otaq kodu:
${currentRoom}); } }
/* ========== 25. DÜYMƏLƏRİN VƏ FORM ELEMENTLƏRİNİN BAĞLANMASI ========== */
function bindButton(id, callback) { const element = $(id);
if (!element) return;
element.addEventListener("click", callback); }
function setupEventListeners() {
// Ana menyu bindButton("createRoom", createRoom); bindButton("joinRoom", joinRoom);
// Qaydalar bindButton("openRules", () => { showScreen("rules"); });
bindButton("closeRules", () => { showScreen("home"); });
// Lobbİ bindButton("copyCode", copyRoomCode); bindButton("readyButton", toggleReady);
bindButton("startGame", startGame); bindButton("leaveLobby", leaveRoom);
// Kart seçimi bindButton("confirmCards", confirmCards);
// Personaj seçimi const characterSelect = $("characterSelect");
if (characterSelect) { characterSelect.addEventListener("change", event => {
changeCharacter(event.target.value); }); }
// Kart düymələri document.querySelectorAll(".game-card").forEach(button => {
button.addEventListener("click", () => { const cardId = button.dataset.card;
if (cardId) { toggleCard(cardId); } }); });
// Enter ilə otağa qoşulmaq const roomInput = $("roomCodeInput");
if (roomInput) { roomInput.addEventListener("keydown", event => { if (event.key === "Enter") {
joinRoom(); } });
roomInput.addEventListener("input", event => { event.target.value = event.target.value
.toUpperCase() .replace(/[^A-Z0-9]/g, "") .slice(0, 4); }); }
// Enter ilə otaq yaratmaq const nicknameInput = $("nickname");
if (nicknameInput) { nicknameInput.addEventListener("keydown", event => { if (event.key ===
"Enter") { createRoom(); } }); } }
/* ========== 26. TƏHLÜKƏSİZ BAŞLANĞIC ========== */
function initializeGame() { setupEventListeners();
const connection = $("connection");
if (connection) { connection.textContent = "● SERVERƏ BAĞLANIR"; }
showScreen("home");
console.log( "%cMASA ARXASINDA", "color:#ffd447;font-size:22px;font-weight:bold;" );
console.log( "%cGame Engine — Part 1 loaded", "color:#54e39a;font-size:13px;" ); }
if (document.readyState === "loading") { document.addEventListener( "DOMContentLoaded",
initializeGame, { once: true } ); } else { initializeGame(); }
/* ========== 1-Cİ HİSSƏ
```

```
BURADA BİTİR. ========== */
/* ========== MASA ARXASINDA — GAME.JS PART 2 — OYUN MEXANİKASI Bu hissəni PART 1-in sonuna əlavə et. ========== */
/* ========== OYUN DƏYİŞƏNLƏRİ ========== */
const CARDS = Object.values(CARD_DATA);
let selectionInterval = null; let faceInterval = null; let practiceState = null; let practiceInterval = null; let currentSwapRequest = null; let isShooting = false;
/* ========== KÖMƏKÇİ FUNKSİYALAR ========== */
function shuffleArray(array) { const result = [...array];
for (let i = result.length - 1; i > 0; i--) { const j = Math.floor(Math.random() * (i + 1)); [result[i], result[j]] = [result[j], result[i]]; }
return result; }
function getShotLimit(playerCount) { if (playerCount <= 3) return 2; if (playerCount <= 5) return 3; if (playerCount <= 7) return 4; return 5; }
function getCurrentGameState() { if (isPracticeMode && practiceState) { return practiceState; }
return roomData?.gameState || null; }
function getCurrentPlayers() { if (isPracticeMode && practiceState) { return practiceState.players; }
return roomData?.players || {}; }
function getCurrentPlayerId() { return isPracticeMode ? "practice-player" : uid; }
function getCurrentPlayer() { return getCurrentPlayers()[getCurrentPlayerId()] || null; }
function getNextAlivePlayer(players, currentId) { const ids = Object.keys(players); const alive = ids.filter(id => players[id]?.alive);
if (alive.length <= 1) return alive[0] || null;
const currentIndex = ids.indexOf(currentId);
for (let i = 1; i <= ids.length; i++) { const nextId = ids[(currentIndex + i) % ids.length];
if (players[nextId]?.alive) { return nextId; } }
return null; }
function getAliveCount(players) { return Object.values(players).filter(p => p?.alive).length; }
function getPlayerDisplayName(player) { return player?.nickname || player?.name || "Naməlum"; }
function getCardInfo(cardId) { return CARDS.find(card => card.id === cardId); }
function hasUsableCard(player, cardId) { return ( Array.isArray(player?.selectedCards) && player.selectedCards.includes(cardId) && !(player.usedCards || []).includes(cardId) ); }
/* ========== EVENTLƏR ========== */
async function publishGameEvent(message, type = "normal") { if (isPracticeMode && practiceState) { practiceState.events = practiceState.events || [];
practiceState.events.unshift({ id: randomId(), message, type, time: Date.now() });
practiceState.events = practiceState.events.slice(0, 30); renderPracticeGame(); return; }
if (!currentRoom) return;
const event = { id: randomId(), message, type, time: Date.now() };
try { await set( ref(db, rooms/currentRoom/events/{event.id}), event ); } catch (error) { console.error("Event yazılmadı:", error); } }
/* ========== RAUND MEXANİKASI ========== */
async function startRoundEngine() { if (isPracticeMode) { startPracticeRound(); return; }
if (!isHost() || !roomData) return;
```

```
const state = roomData.gameState; if (!state || state.phase !== "cardSelection") return;
const players = roomData.players || {}; const playerIds = Object.keys(players);
if (playerIds.length < 2) return;
const allReady = playerIds.every(id => { return players[id].cardReady === true; });
const config = getRoundConfig(state.round);
if (!allReady && !config.automaticCards) return;
const chamberArray = Array(config.chambers).fill(false);
let placed = 0;
while (placed < config.bullets) { const index = Math.floor(Math.random() * chamberArray.length);
if (!chamberArray[index]) { chamberArray[index] = true; placed++; } }
const shuffledChambers = shuffleArray(chamberArray);
const updates = {};
for (const id of playerIds) { const player = players[id];
let cards = player.selectedCards || [];
if (config.automaticCards) { cards = shuffleArray(CARDS.map(card => card.id)).slice( 0,
config.cards ); }
updates[players/id/selectedCards] = cards; updates[players/{id}/usedCards] = [];
updates[players/${id}/cardReady] = false; }
const newState = { ...state, phase: "playing", chambers: shuffledChambers, chamberIndex: 0,
bulletsLeft: config.bullets, shotsTaken: 0, shotLimit: getShotLimit(playerIds.length), currentTurn:
state.firstTurn || playerIds[0], pendingSwap: null, startedAt: Date.now() };
updates.gameState = newState;
try { await update(getRoomRef(), updates); await publishGameEvent( ${state.round}-ci raund
başladı!, "round" ); } catch (error) { showError(error.message); } }
async function beginNextRound() { if (isPracticeMode) { startPracticeRound(); return; }
if (!isHost() || !roomData) return;
const state = roomData.gameState; if (!state || state.phase !== "roundEnd") return;
const nextRound = state.round + 1; const players = roomData.players || {}; const aliveIds =
Object.keys(players).filter( id => players[id]?.alive );
if (aliveIds.length <= 1) { await finishGame(); return; }
if (nextRound > 5) { await finishGame(); return; }
const config = getRoundConfig(nextRound); const updates = {};
for (const id of aliveIds) { updates[players/id/cardReady] = false; updates[players/
{id}/selectedCards] = []; updates[players/${id}/usedCards] = []; }
updates.gameState = { round: nextRound, phase: "cardSelection", chambers: [], chamberIndex:
0, bulletsLeft: config.bullets, shotsTaken: 0, shotLimit: getShotLimit(aliveIds.length), currentTurn:
aliveIds[0], firstTurn: aliveIds[0], pendingSwap: null, selectionEndsAt: config.selectionTime ?
Date.now() + config.selectionTime * 1000 : null };
try { await update(getRoomRef(), updates); await publishGameEvent( ${nextRound}-ci raund
üçün kart seçimi başladı!, "round" ); } catch (error) { showError(error.message); } }
/* ========================= KART SEÇİMİ SAYĞACI =========================
*/
function startSelectionTimer() { clearInterval(selectionInterval);
if (isPracticeMode || !roomData) return;
const state = roomData.gameState; if (state?.phase !== "cardSelection") return;
const config = getRoundConfig(state.round);
if (!config.selectionTime) { if (isHost()) startRoundEngine(); return; }
selectionInterval = setInterval(async () => { const currentState = roomData?.gameState;
```

```
if (!currentState || currentState.phase !== "cardSelection") { clearInterval(selectionInterval);
return; }
const remaining = Math.max( 0, Math.ceil((currentState.selectionEndsAt - Date.now()) / 1000) );
setText("cardTimer", ${remaining} saniyə);
if (remaining <= 0 && isHost()) { clearInterval(selectionInterval);
const players = roomData.players || {}; const updates = {};
for (const [id, player] of Object.entries(players)) { if (player.cardReady) continue;
let cards = player.selectedCards || [];
while (cards.length < config.cards) { const available = CARDS .map(card => card.id)
.filter(cardId => !cards.includes(cardId));
if (!available.length) break;
cards.push(randomItem(available)); }
updates[players/id/selectedCards] = cards; updates[players/{id}/cardReady] = true; }
if (Object.keys(updates).length) { await update(getRoomRef(), updates); }
setTimeout(() => startRoundEngine(), 500); } }, 500); }
/* ========================= OYUN EKRANININ RENDERİ
========================= */
function renderGameScreen() { const state = getCurrentGameState(); const players =
getCurrentPlayers();
if (!state || !players) return;
const alivePlayers = Object.entries(players).filter( ([id, player]) => player?.alive );
setText("gameRoomCode", currentRoom || "MƏŞQ"); setText("aliveCount", alivePlayers.length);
setText("roundLabel", state.round); setText("chamberCount", state.chambers?.length || 0);
setText("bulletsLeft", state.bulletsLeft ?? 0); setText("shotsTaken", state.shotsTaken ?? 0);
setText("shotLimit", state.shotLimit ?? 0);
const currentTurnPlayer = players[state.currentTurn];
setText( "turnLabel", currentTurnPlayer ? ${getPlayerDisplayName(currentTurnPlayer)} oynayır :
"Növbə gözlənilir" );
renderSeats(); renderActiveCards(); renderLatestEvent(); renderEventLog();
const myTurn = state.currentTurn === getCurrentPlayerId(); const canShoot = state.phase ===
"playing" && myTurn && !isShooting && !state.pendingSwap && state.shotsTaken <
state.shotLimit;
const shootButton = $("shootButton");
if (shootButton) { shootButton.disabled = !canShoot; shootButton.textContent = canShoot ?
"ATƏŞ AÇ" : "NÖVBƏNİ GÖZLƏ"; }
showScreen("game"); }
function renderSeats() { const seatList = $("seatList"); if (!seatList) return;
const players = getCurrentPlayers(); const state = getCurrentGameState();
seatList.innerHTML = "";
const entries = Object.entries(players); const alive = entries.filter(([id, player]) => player?.alive);
entries.forEach(([id, player], index) => { const character = getCharacter(player.character); const
isCurrent = state?.currentTurn === id;
const seat = document.createElement("div"); seat.className = seat ${player.alive ? "" :
"eliminated"} ${ isCurrent ? "current-turn" : "" };
seat.innerHTML = <div class="face"> ${character?.face || "👤"} </div> <div class="seat-name">
${escapeHTML(getPlayerDisplayName(player))} </div> <div class="seat-hearts">
${"❤️".repeat(Math.max(0, player.hearts || 0))} </div> <div class="seat-status"> ${player.alive ?
"OYUNDA" : "ƏLƏNİB"} </div>;
```

```
seat.dataset.playerId = id;
if (player.alive && id !== getCurrentPlayerId()) { seat.addEventListener("click", () => { if
(state?.currentTurn !== getCurrentPlayerId()) return; if (state?.phase !== "playing") return;
selectedTarget = id; highlightTarget(id); }); }
seatList.appendChild(seat); }); }
function highlightTarget(targetId) { document.querySelectorAll(".seat").forEach(seat => {
seat.classList.toggle( "selected-target", seat.dataset.playerId === targetId ); });
const target = getCurrentPlayers()[targetId];
if (target) { setText( "shotResult", Hədəf: ${getPlayerDisplayName(target)} ); } }
function renderActiveCards() { const container = $("activeCards"); if (!container) return;
const player = getCurrentPlayer(); const state = getCurrentGameState();
container.innerHTML = "";
if (!player || state?.phase !== "playing") return;
(player.selectedCards || []).forEach(cardId => { const card = getCardInfo(cardId); if (!card)
return;
const used = (player.usedCards || []).includes(cardId);
const button = document.createElement("button"); button.className = active-card ${used ?
"used" : ""}; button.disabled = used; button.textContent = card.name{used ? " — İSTİFADƏ
OLUNUB" : ""};
button.addEventListener("click", () => activateCard(cardId));
container.appendChild(button); }); }
/* ========================= KARTLARIN AKTİVLƏŞDİRİLMƏSİ
========================= */
async function activateCard(cardId) { const state = getCurrentGameState(); const player =
getCurrentPlayer();
if (!state || !player) return; if (state.phase !== "playing") return; if (state.currentTurn !==
getCurrentPlayerId()) return; if (!hasUsableCard(player, cardId)) return;
if (cardId === "detective") { const nextBullet = state.chambers?.[state.chamberIndex];
if (nextBullet === undefined) { showToast("Daraqda güllə qalmayıb."); return; }
await markCardUsed(cardId);
showToast( nextBullet ? "Detektiv: Növbəti kamerada GÜLLƏ var." : "Detektiv: Növbəti kamera
BOŞDUR." );
await publishGameEvent( ${getPlayerDisplayName(player)} Detektiv kartından istifadə etdi.,
"card" );
return; }
if (cardId === "counter") { showToast("Əks təsir aktivdir. Növbəti güllələ zərbəni bloklayacaq.");
await markCardUsed(cardId); await publishGameEvent( ${getPlayerDisplayName(player)} Əks
təsir kartını aktivləşdirdi., "card" ); return; }
if (cardId === "witch") { showToast("İfritəlik aktivdir: növbəti atəşin dəymə ehtimalı 80%-dir.");
await markCardUsed(cardId); await setTemporaryEffect("witch"); return; }
if (cardId === "selfLove") { showToast("Canını sevmək aktivdir: özünə atəş açanda dəymə
ehtimalı 25%-dir."); await markCardUsed(cardId); await setTemporaryEffect("selfLove"); return; }
if (cardId === "swap") { showToast("Yerdəyişmə kartı aktivdir. Sənə atəş açılanda seçim edə
biləcəksən."); await markCardUsed(cardId); await setTemporaryEffect("swap"); return; }
if (cardId === "evil") { showToast("İbnəlik aktivdir: növbəti atəşdə xüsusi nəticə tətbiq
olunacaq."); await markCardUsed(cardId); await setTemporaryEffect("evil"); return; } }
async function markCardUsed(cardId) { if (isPracticeMode) { const player = getCurrentPlayer();
player.usedCards = player.usedCards || [];
```

```
if (!player.usedCards.includes(cardId)) { player.usedCards.push(cardId); }
renderPracticeGame(); return; }
const player = getMyPlayer(); if (!player) return;
const used = player.usedCards || [];
if (!used.includes(cardId)) { await update( ref(db, rooms/currentRoom/players/{uid}), {
usedCards: [...used, cardId] } ); } }
async function setTemporaryEffect(effect) { if (isPracticeMode) { practiceState.effects =
practiceState.effects || {}; practiceState.effects[getCurrentPlayerId()] =
practiceState.effects[getCurrentPlayerId()] || {};
practiceState.effects[getCurrentPlayerId()][effect] = true; renderPracticeGame(); return; }
const effects = roomData.gameState.effects || {}; const myEffects = effects[uid] || {};
await update(getRoomRef(), { [gameState/effects/uid/{effect}]: true }); }
/* ======================== ATƏŞ AÇMA ======================== */
async function shootTarget(targetId) { if (isShooting) return;
const state = getCurrentGameState(); const players = getCurrentPlayers(); const shooterId =
getCurrentPlayerId();
if (!state || state.phase !== "playing") return; if (state.currentTurn !== shooterId) return; if
(!players[targetId]?.alive) return; if (targetId === shooterId && !hasSelfTargetPermission()) {
showToast("Özünə atəş açmaq üçün Canını sevmək kartı lazımdır."); return; }
if (state.shotsTaken >= state.shotLimit) { showToast("Bu raund üçün atəş limitin bitib."); return; }
if (state.chamberIndex >= state.chambers.length) { showToast("Daraq boşalıb."); return; }
isShooting = true;
const chamberLoaded = state.chambers[state.chamberIndex];
const shooter = players[shooterId]; const target = players[targetId];
const effects = state.effects || {}; const shooterEffects = effects[shooterId] || {};
let hitChance = chamberLoaded ? 1 : 0; let forcedHit = null;
if (shooterEffects.witch) { hitChance = 0.8; }
if (targetId === shooterId && shooterEffects.selfLove) { hitChance = 0.25; }
if (shooterEffects.witch || shooterEffects.selfLove) { forcedHit = Math.random() < hitChance; }
else { forcedHit = chamberLoaded; }
const isEvil = !!shooterEffects.evil;
const result = { shooterId, targetId, loaded: chamberLoaded, hit: forcedHit, evil: isEvil, time:
Date.now() };
await playShotAnimation(targetId);
if (isPracticeMode) { await resolvePracticeShot(result); } else { await resolveOnlineShot(result); }
isShooting = false; }
function hasSelfTargetPermission() { const player = getCurrentPlayer();
return ( player && (player.selectedCards || []).includes("selfLove") ); }
async function playShotAnimation(targetId) { const gun = $("tableGun"); const effect =
$("shotEffect");
const targetSeat = document.querySelector( .seat[data-player-id="${targetId}"] );
if (gun && targetSeat) { gun.classList.add("aiming"); }
await new Promise(resolve => setTimeout(resolve, 600));
if (effect) { effect.classList.add("active"); }
const blackout = $("blackout");
if (blackout) { blackout.classList.add("active"); }
await new Promise(resolve => setTimeout(resolve, 900));
if (blackout) { blackout.classList.remove("active"); }
```

```
if (effect) { effect.classList.remove("active"); }
if (gun) { gun.classList.remove("aiming"); } }
/* ========================= ONLAYN ATƏŞİN HƏLLİ ========================= */
async function resolveOnlineShot(result) { if (!isHost()) { showToast("Atəş nəticəsini otaq sahibi təsdiqləyir."); return; }
const state = roomData.gameState; const players = roomData.players || {};
if (state.currentTurn !== result.shooterId) return;
const shooter = players[result.shooterId]; const target = players[result.targetId];
if (!shooter || !target) return;
const nextIndex = state.chamberIndex + 1; const nextShots = state.shotsTaken + 1;
const updates = {};
updates["gameState/chamberIndex"] = nextIndex; updates["gameState/bulletsLeft"] = Math.max( 0, state.bulletsLeft - (result.loaded ? 1 : 0) ); updates["gameState/shotsTaken"] = nextShots;
updates[gameState/effects/result. shooterId/witch] = null; updates[gameState/effects/{result.shooterId}/selfLove] = null; updates[gameState/effects/${result.shooterId}/evil] = null;
let message = "";
if (result.evil) { if (result.hit) { updates[players/result. targetId/hearts] = 0; updates[players/{result.targetId}/alive] = false;
message = ${getPlayerDisplayName(target)} İbnəlik kartı ilə oyundan çıxarıldı!; } else { const newHearts = Math.max(0, shooter.hearts - 1);
updates[players/${result.shooterId}/hearts] = newHearts;
if (newHearts <= 0) { updates[players/${result.shooterId}/alive] = false; }
message = ${getPlayerDisplayName(shooter)} İbnəlik kartına görə 1 can itirdi.; } } else if (result.hit) { const targetEffects = state.effects?.[result.targetId] || {};
if (targetEffects.counter) { updates[gameState/effects/${result.targetId}/counter] = null;
const newHearts = Math.min(3, target.hearts + 1);
updates[players/${result.targetId}/hearts] = newHearts;
message = ${getPlayerDisplayName(target)} Əks təsir ilə gülləni blokladı və 1 can qazandı!; } else if (targetEffects.swap) { updates["gameState/pendingSwap"] = { shooterId: result.shooterId, targetId: result.targetId, result, createdAt: Date.now() };
await update(getRoomRef(), updates);
showSwapPrompt( result.shooterId, result.targetId ); return; } else { const newHearts = Math.max(0, target.hearts - 1);
updates[players/${result.targetId}/hearts] = newHearts;
updates[players/${result.targetId}/hearts] = newHearts;
if (newHearts <= 0) { updates[players/${result.targetId}/alive] = false; }
message = ${getPlayerDisplayName(target)} güllə dəydi və 1 can itirdi.; } } else { message = "Atəş açıldı, amma güllə boş idi."; }
updates["gameState/pendingSwap"] = null;
const updatedPlayers = structuredClone(players);
if (updates[players/${result.targetId}/hearts] !== undefined) {
updatedPlayers[result.targetId].hearts = updates[players/${result.targetId}/hearts];
updatedPlayers[result.targetId].alive = updatedPlayers[result.targetId].hearts > 0; }
if (updates[players/${result.shooterId}/hearts] !== undefined) {
updatedPlayers[result.shooterId].hearts = updates[players/${result.shooterId}/hearts];
updatedPlayers[result.shooterId].alive = updatedPlayers[result.shooterId].hearts > 0; }
```

```
const aliveCount = getAliveCount(updatedPlayers);
if (aliveCount <= 1) { updates["gameState/phase"] = "gameEnd"; } else if ( nextShots >=
state.shotLimit || nextIndex >= state.chambers.length ) { updates["gameState/phase"] =
"roundEnd"; } else { const nextPlayer = getNextAlivePlayer( updatedPlayers, result.shooterId );
updates["gameState/currentTurn"] = nextPlayer; }
try { await update(getRoomRef(), updates);
await publishGameEvent(message, result.hit ? "danger" : "normal");
if (aliveCount <= 1) { await finishGame(); } } catch (error) { showError(error.message); } }
/* ========================= YERDƏYİŞMƏ ========================= */
function showSwapPrompt(shooterId, targetId) { const modal = $("swapPrompt"); if (!modal)
return;
const shooter = getCurrentPlayers()[shooterId];
setText( "swapMessage", ${getPlayerDisplayName(shooter)} səni hədəfə aldı. Zərbəni ona
qaytarmaq istəyirsən? );
modal.classList.add("active"); }
async function answerSwap(accept) { if (isPracticeMode) { await resolvePracticeSwap(accept);
return; }
const state = roomData?.gameState; const pending = state?.pendingSwap;
if (!pending || pending.targetId !== uid) return;
const modal = $("swapPrompt"); if (modal) modal.classList.remove("active");
const players = roomData.players || {}; const shooter = players[pending.shooterId]; const target
= players[pending.targetId];
const updates = { "gameState/pendingSwap": null,
[gameState/effects/${pending.targetId}/swap]: null };
let message = "";
if (accept) { const newHearts = Math.max(0, shooter.hearts - 1);
updates[players/${pending.shooterId}/hearts] = newHearts;
if (newHearts <= 0) { updates[players/${pending.shooterId}/alive] = false; }
message = ${getPlayerDisplayName(target)} Yerdəyişmə etdi!
${getPlayerDisplayName(shooter)} 1 can itirdi.; } else { const newHearts = Math.max(0,
target.hearts - 1);
updates[players/${pending.targetId}/hearts] = newHearts;
if (newHearts <= 0) { updates[players/${pending.targetId}/alive] = false; }
message = ${getPlayerDisplayName(target)} Yerdəyişmədən imtina etdi və 1 can itirdi.; }
if ( state.shotsTaken >= state.shotLimit || state.chamberIndex >= state.chambers.length ) {
updates["gameState/phase"] = "roundEnd"; } else { updates["gameState/currentTurn"] =
getNextAlivePlayer( players, pending.shooterId ); }
await update(getRoomRef(), updates); await publishGameEvent(message, "danger"); }
/* ========================= RAUND / OYUN SONU ========================= */
async function finishGame() { if (isPracticeMode) { const alive =
Object.values(practiceState.players).filter( player => player.alive );
practiceState.phase = "gameEnd"; practiceState.winner = alive[0] || null;
renderPracticeEnd(); return; }
if (!isHost() || !roomData) return;
const players = roomData.players || {}; const alive = Object.entries(players).filter( ([id, player])
=> player.alive );
const winner = alive.length === 1 ? alive[0][1] : null;
await update(getRoomRef(), { "gameState/phase": "gameEnd", "gameState/winner": winner ?
```

```
getPlayerDisplayName(winner) : "Heç kim" });
showScreen("ended");
setText( "winnerTitle", winner ? ${getPlayerDisplayName(winner)} QALİB OLDU! : "OYUN
BİTDİ" );
setText( "winnerText", winner ? "Masa arxasında sonuncu sağ qalan oyunçu!" : "Bu oyunda qalib
müəyyən olunmadı." ); }
function renderPracticeEnd() { const winner = practiceState.winner;
showScreen("ended");
setText( "winnerTitle", winner ? ${getPlayerDisplayName(winner)} QALİB OLDU! : "OYUN
BİTDİ" );
setText( "winnerText", winner ? "Məşq rejiminin qalibi!" : "Heç kim sağ qalmadı." ); }
/* ========================= MƏŞQ REJİMİ — BOTLAR
========================= */
function startPracticeMode() { const nickname = getNickname(); const count =
Number($("botCount")?.value || 3);
if (!nickname) { showToast("Əvvəlcə adını yaz."); return; }
isPracticeMode = true; currentRoom = null;
const human = { nickname, character: CHARACTERS[0].id, hearts: 2, alive: true,
selectedCards: [], usedCards: [], cardReady: false, isBot: false };
const players = { "practice-player": human };
for (let i = 0; i < count; i++) { const character = CHARACTERS[(i + 1) % CHARACTERS.length];
players[bot-${i + 1}] = { nickname: BOT ${i + 1}, character: character.id, hearts: 2, alive: true,
selectedCards: [], usedCards: [], cardReady: false, isBot: true }; }
practiceState = { round: 1, phase: "cardSelection", players, chambers: [], chamberIndex: 0,
bulletsLeft: 2, shotsTaken: 0, shotLimit: getShotLimit(count + 1), currentTurn: "practice-player",
effects: {}, events: [], pendingSwap: null, winner: null };
showScreen("cardSelection"); renderPracticeCardSelection(); }
function renderPracticeCardSelection() { const config = getRoundConfig(practiceState.round);
setText("cardRound", practiceState.round); setText("cardsToChoose", config.cards); setText(
"cardSelectionDescription", Məşq rejimi — ${config.cards} kart seç. );
selectedCards = [];
document.querySelectorAll(".game-card").forEach(button => {
button.classList.remove("selected"); button.disabled = false; });
setText("selectedCardCount", 0 / ${config.cards}); }
function startPracticeRound() { if (!practiceState) return;
const config = getRoundConfig(practiceState.round); const chamberArray =
Array(config.chambers).fill(false);
let placed = 0;
while (placed < config.bullets) { const index = Math.floor(Math.random() * chamberArray.length);
if (!chamberArray[index]) { chamberArray[index] = true; placed++; } }
practiceState.chambers = shuffleArray(chamberArray); practiceState.chamberIndex = 0;
practiceState.bulletsLeft = config.bullets; practiceState.shotsTaken = 0; practiceState.shotLimit
= getShotLimit( Object.keys(practiceState.players).length ); practiceState.phase = "playing";
practiceState.effects = {};
for (const [id, player] of Object.entries(practiceState.players)) { if (!player.alive) continue;
player.usedCards = [];
if (id !== "practice-player") { player.selectedCards = shuffleArray( CARDS.map(card => card.id)
).slice(0, config.cards); } }
```

```
practiceState.currentTurn = "practice-player";
showScreen("game"); renderPracticeGame(); publishGameEvent(${practiceState.round}-ci
raund başladı!); }
function renderPracticeGame() { if (!practiceState) return;
renderGameScreen();
const events = practiceState.events || [];
setText( "latestEvent", events[0]?.message || "Oyun başladı." );
const eventLog = $("eventLog");
if (eventLog) { eventLog.innerHTML = events .slice(0, 15) .map(event => <div
class="event-item">${escapeHTML(event.message)}</div>) .join(""); } }
async function resolvePracticeShot(result) { const players = practiceState.players; const shooter
= players[result.shooterId]; const target = players[result.targetId];
practiceState.chamberIndex++; practiceState.shotsTaken++;
if (result.loaded) { practiceState.bulletsLeft--; }
const effects = practiceState.effects || {}; const shooterEffects = effects[result.shooterId] || {};
const targetEffects = effects[result.targetId] || {};
shooterEffects.witch = false; shooterEffects.selfLove = false; shooterEffects.evil = false;
let message = "";
if (result.evil) { if (result.hit) { target.hearts = 0; target.alive = false;
message = ${target.nickname} İbnəlik kartı ilə oyundan çıxdı!; } else { shooter.hearts =
Math.max(0, shooter.hearts - 1);
if (shooter.hearts <= 0) shooter.alive = false;
message = ${shooter.nickname} 1 can itirdi.; } } else if (result.hit) { if (targetEffects.counter) {
targetEffects.counter = false; target.hearts = Math.min(3, target.hearts + 1);
message = ${target.nickname} Əks təsir ilə qorundu və 1 can qazandı!; } else if
(targetEffects.swap) { practiceState.pendingSwap = { shooterId: result.shooterId, targetId:
result.targetId };
currentSwapRequest = practiceState.pendingSwap; showSwapPrompt(result.shooterId,
result.targetId); renderPracticeGame(); return; } else { target.hearts = Math.max(0, target.hearts
- 1);
if (target.hearts <= 0) target.alive = false;
message = ${target.nickname} güllə dəydi və 1 can itirdi.; } } else { message = "Güllə boş idi."; }
await publishGameEvent(message, result.hit ? "danger" : "normal");
const aliveCount = getAliveCount(players);
if (aliveCount <= 1) { finishGame(); return; }
if ( practiceState.shotsTaken >= practiceState.shotLimit || practiceState.chamberIndex >=
practiceState.chambers.length ) { practiceState.phase = "roundEnd";
renderPracticeRoundEnd(); return; }
practiceState.currentTurn = getNextAlivePlayer( players, result.shooterId );
renderPracticeGame();
if (practiceState.currentTurn.startsWith("bot-")) { setTimeout(botTurn, 1200); } }
async function resolvePracticeSwap(accept) { const pending = practiceState?.pendingSwap; if
(!pending) return;
const shooter = practiceState.players[pending.shooterId]; const target =
practiceState.players[pending.targetId];
if (accept) { shooter.hearts = Math.max(0, shooter.hearts - 1);
if (shooter.hearts <= 0) shooter.alive = false;
await publishGameEvent( ${target.nickname} Yerdəyişmə etdi. ${shooter.nickname} 1 can itirdi.,
```

```
"danger" ); } else { target.hearts = Math.max(0, target.hearts - 1);
if (target.hearts <= 0) target.alive = false;
await publishGameEvent( ${target.nickname} Yerdəyişmədən imtina etdi və 1 can itirdi.,
"danger" ); }
practiceState.pendingSwap = null; currentSwapRequest = null;
const modal = $("swapPrompt"); if (modal) modal.classList.remove("active");
const aliveCount = getAliveCount(practiceState.players);
if (aliveCount <= 1) { finishGame(); return; }
if ( practiceState.shotsTaken >= practiceState.shotLimit || practiceState.chamberIndex >=
practiceState.chambers.length ) { practiceState.phase = "roundEnd";
renderPracticeRoundEnd(); return; }
practiceState.currentTurn = getNextAlivePlayer( practiceState.players, pending.shooterId );
renderPracticeGame();
if (practiceState.currentTurn.startsWith("bot-")) { setTimeout(botTurn, 1200); } }
/* ========================= BOT SİSTEMİ ========================= */
function botTurn() { if (!practiceState || practiceState.phase !== "playing") return;
const botId = practiceState.currentTurn;
if (!botId || !botId.startsWith("bot-")) return;
const bot = practiceState.players[botId];
if (!bot || !bot.alive) return;
const aliveTargets = Object.entries(practiceState.players) .filter(([id, player]) => id !== botId &&
player.alive);
if (!aliveTargets.length) return;
const [targetId, target] = randomItem(aliveTargets);
const effects = practiceState.effects[botId] || {}; practiceState.effects[botId] = effects;
if ( bot.selectedCards.includes("detective") && !bot.usedCards.includes("detective") ) {
bot.usedCards.push("detective");
publishGameEvent( ${bot.nickname} Detektiv kartından istifadə etdi. ); }
if ( bot.selectedCards.includes("counter") && !bot.usedCards.includes("counter") && bot.hearts
<= 1 ) { bot.usedCards.push("counter"); effects.counter = true; }
if ( bot.selectedCards.includes("witch") && !bot.usedCards.includes("witch") ) {
bot.usedCards.push("witch"); effects.witch = true; }
if ( bot.selectedCards.includes("evil") && !bot.usedCards.includes("evil") ) {
bot.usedCards.push("evil"); effects.evil = true; }
const loaded = practiceState.chambers[ practiceState.chamberIndex ];
let hit = loaded;
if (effects.witch) { hit = Math.random() < 0.8; }
const result = { shooterId: botId, targetId, loaded, hit, evil: !!effects.evil, time: Date.now() };
playShotAnimation(targetId).then(() => { resolvePracticeShot(result); }); }
/* ========================= MƏŞQ RAUNDUNUN SONU
========================= */
function renderPracticeRoundEnd() { const alive = Object.values(practiceState.players).filter(
player => player.alive );
if (alive.length <= 1) { finishGame(); return; }
setText( "latestEvent", ${practiceState.round}-ci raund başa çatdı. );
setText("turnLabel", "Raund bitdi");
const shootButton = $("shootButton");
if (shootButton) { shootButton.disabled = false; shootButton.textContent = "NÖVBƏTİ RAUND";
```

```
shootButton.onclick = () => { practiceState.round++;
if (practiceState.round > 5) { finishGame(); } else { renderPracticeCardSelection();
showScreen("cardSelection"); } }; }
renderPracticeGame(); }
/* ========================= ATƏŞ DÜYMƏSİ ========================= */
function handleShootButton() { const state = getCurrentGameState();
if (!state || state.phase !== "playing") return;
if (state.currentTurn !== getCurrentPlayerId()) { showToast("Hələ sənin növbən deyil."); return; }
if (!selectedTarget) { showToast("Əvvəlcə hədəf oyunçunu seç."); return; }
shootTarget(selectedTarget); }
/* ========================= ÜZ ANİMASİYALARI ========================= */
function startFaceAnimations() { clearInterval(faceInterval);
faceInterval = setInterval(() => { document.querySelectorAll(".face").forEach(face => {
face.classList.add("blink");
setTimeout(() => { face.classList.remove("blink"); }, 180); }); }, 3000); }
/* ========================= OYUN EKRANININ YENİLƏNMƏSİ
========================= */
function renderRoom() { if (isPracticeMode) { renderPracticeGame(); return; }
if (!roomData) return;
const state = roomData.gameState; if (!state) return;
if (state.phase === "cardSelection") { showScreen("cardSelection"); renderCardSelection();
startSelectionTimer(); }
if (state.phase === "playing") { clearInterval(selectionInterval); renderGameScreen(); }
if (state.phase === "roundEnd") { renderGameScreen();
const button = $("shootButton");
if (button) { button.disabled = false; button.textContent = "NÖVBƏTİ RAUND"; button.onclick = ()
=> { if (isHost()) beginNextRound(); else showToast("Növbəti raundu otaq sahibi başladacaq.");
}; } }
if (state.phase === "gameEnd") { showScreen("ended");
setText( "winnerTitle", ${state.winner || "Oyun"} QALİB OLDU! );
setText( "winnerText", "Masa arxasında oyun başa çatdı." ); }
if (state.pendingSwap?.targetId === uid) { showSwapPrompt( state.pendingSwap.shooterId,
state.pendingSwap.targetId ); } }
/* ========================= DÜYMƏLƏR ========================= */
function setupGameControls() { const shootButton = $("shootButton");
if (shootButton) { shootButton.addEventListener("click", handleShootButton); }
const acceptSwap = $("acceptSwap");
if (acceptSwap) { acceptSwap.addEventListener("click", () => answerSwap(true)); }
const declineSwap = $("declineSwap");
if (declineSwap) { declineSwap.addEventListener("click", () => answerSwap(false)); }
const practiceButton = $("practiceButton");
if (practiceButton) { practiceButton.addEventListener("click", startPracticeMode); }
const backHome = $("backHome");
if (backHome) { backHome.addEventListener("click", () => { isPracticeMode = false;
practiceState = null; currentRoom = null; selectedTarget = null;
clearInterval(selectionInterval); clearInterval(faceInterval);
showScreen("home"); }); } }
/* ========================= SON İŞƏ SALMA ========================= */
```

```
setupGameControls(); startFaceAnimations();
/* ========================= PART 2 SONU ========================= */
```